# DPLL SAT-solver

liseej@uio.no

October 26, 2026

## 1 Introduction

This report explains the design and implementation of a SAT solver: a program that checks whether a propositional formula in Conjunctive Normal Form (CNF) is satisfiable with its satisfying interpretation. The program take the user input in the from a `.txt` file. We have two test cases: 8 clauses that can be built from 3 propositional variables, and test how large of a $n$ the program can handle. The project is written in Haskell and uses a Nix flake to build and run it. For more information and how to run the project, see the README.md file in the project root.

## 2 Theory

This is a DPLL solver and uses: 1) **unit propagation**: if a clause has only one literal L, then L must be true. Remove every clause containing L, and remove the complement of L from every other clause. 2) **atomic cut**: pick an atom `a` and split into two branches, one where `a` is true and one where it's false.

The DPLL proof works on a one-sided sequent: all clauses on the left, nothing on the right:

$$C_1, C_2, \ldots, C_n \Rightarrow$$

- A branch *closes* when the empty clause $\square$ appears. Since $\square$ means "false", the branch has no model. *Unsatisfiable*: every branch closes.
- A branch is *open* when no clauses are left. Every clause was satisfied by the literals assumed on that branch, and these literals then form a model. *Satisfiable*: some branch is open.

## 3 Design and Implementation

### 3.1 Syntax: How formulas are represented

**Literals.** Literals are represented as integers, i.e., 1 for p, -1 for $\neg p$ and so on. The complement of a literal is just its negation. 0 is never used as an atom, since its negation would also be 0. So, the check for positive literals, is to check if the atoms $> 0$. There is no need to pattern match on `Not`. To find an atom of a literal, it's the absolute value.

**Clauses.** The solver only asks two things: "does it contain L " and "remove $\neg L$ from it". A clause is represented as a set of integers. As it is a set, order and duplication do not matter. With the use of `IntSet`, finding and deleting a literal is fast. To represent multiple clauses (in CNF), we use a plain list of clauses. At each step, the solver builds a new clause set (a set of integers), and it traverses the list (the collection of all clauses on a branch) once. An `IntSet` also keeps its elements sorted, which the solver uses when choosing an atom to cut on (see Section 3.2).

**Model.** A model is the name given to a list of literals, with the newest first. It records the literals assumed true on the current branch. A literal is added to the model in two ways: it is forced by unit propagation, or it is chosen by a cut, which splits the search into two branches. Example: if we assume L, then $\neg L$ is removed from every clause, so that it never can become a unit clause later. If $\neg L$ was itself a unit clause, removing it gives the empty clause $\square$, and the branch is closed.

**Sequent.** A sequent is the clause set at a node of the proof tree, written $C_1, \ldots, C_n \Rightarrow$. Since we never build the whole tree, the solver only holds the sequent at the current end of the branch, the *leaf sequent*. For each branch tow parts is kept as separate arguments:

1. the clauses still left after propagation and cut, which form the leaf sequent;
2. the literals assumed so far, which form the model.

**User input (`parseCNF`).** How the program read user input from a `.txt` file.

Each line is one clause, with the literals written as integers separated by spaces, so `1 -2 3` is $\{p, \neg q, r\}$. A clause can have any number of literals. Lines starting with `c` are comments, and a trailing `0` is ignored. If a literal is not an integer, `parseCNF` rejects the file instead of guessing.

**Empty clause and empty clause set.** The solver works with two empty objects that mean opposite things. The empty clause $\square$ is a disjunction with no literals, so it is false and closes the branch. The branch gives no model, and the solver stops searching it. The empty clause set is a conjunction with no clauses, so it is true and leaves the branch open with a model. This means that the original clause set is satisfiable, and the model tells us under which interpretation: each literal assumed on the branch gives its atom a value, true for a positive literal and false for a negative one. Atoms the solver never assigned get the value false (completeModel). Any value works, since no remaining clause contains them.

## 3.2 The DPLL algorithm

We search the proof tree one branch at a time. Each branch holds the clauses still left, and the literals assumed true so far. A branch either closes, giving no model, or ends open, giving a model. We never build the whole tree, since building a full proof tree is complicated, slow, and may take a lot of memory.

**Simplification (`simplify`).** Assuming a literal $L$ is true turns the clause set $S$ into $S[L]$. Clauses containing $L$ are true and are removed (unit subsumption). From clauses containing $\neg L$, we remove $\neg L$ (unit resolution). All other clauses stay unchanged. If the last literal of a clause is removed, the empty clause $\square$ appears.

**Unit propagation (`propagate`).** We first check for $\square$, so that a closed branch stops as early as possible. If there is no $\square$, we look for a unit clause $\{L\}$, add $L$ to the model, and simplify with $L$. We repeat until no unit clause is left, because one simplification can create a new unit clause. The process always ends, since every round removes at least one clause.

**Atomic cut (`chooseAtom` and `dpll`).** If clauses remain after propagation, we look at the first clause. Since clauses are kept sorted, its first literal is the smallest one, and we cut on the atom of that literal: the clause $\{p, \neg q, r\}$ is stored as $\{-2, 1, 3\}$, so the smallest literal is $-2$ and we cut on $q$. We try the atom as true first. Only if that branch closes do we try it as false. If both branches close, this part of the tree is closed. Haskell evaluates lazily, so the second branch is never started when the first one finds a model. Values are immutable, so backtracking needs no undo step: the second branch starts from the clause set and model as they were before the cut.

**Correctness (`solve`).** Simplifying with $L$ keeps exactly the models in which $L$ is true. A unit clause $\{L\}$ forces $L$ in every model. Every interpretation makes an atom either true or false, so if both cut branches close, $S$ has no model. On an open branch, every clause was removed because it contained a literal that is true in the model, so the model satisfies all clauses. Solve returns Unsatisfiable if every branch closes, and otherwise Satisfiable with the completed model.

**Behaviour on $S_n$.** $S_n$ contains no unit clauses at the start, so the solver cuts $n-1$ times. Unit propagation then closes each branch, which saves the last level of cuts. The tree has $2^{n-1}$ closed leaves, and each level costs about $2^n$ work, giving roughly $n \cdot 2^n$ in total.

**Memory.** Each pending second branch keeps the clause set from just before its cut. On $S_n$, these saved sets together hold about as many clauses as the input. We also keep the input until the end, because we need it to complete the model. Since the input doubles with every n, memory grows as fast as time. At $n = 24$ the program needs 7 GB, so $n = 25$ would not fit in a 8 GB cap.

# 4 Result

## Case 1

The first set contains all 8 clauses that can be built from the three propositional variables $p, q, r$ and their negations:

$$S_3 = \{\, \{p, q, r\}, \{p, q, \neg r\}, \{p, \neg q, r\}, \{p, \neg q, \neg r\}, \\ \{\neg p, q, r\}, \{\neg p, q, \neg r\}, \{\neg p, \neg q, r\}, \{\neg p, \neg q, \neg r\} \,\}$$

The solver returns *UNSAT* for $S_3$, as expected. The Table 1 shows the result after deleting one clause and its satisfying interpretation.

| Removed clause | $p$ | $q$ | $r$ |
|---|---|---|---|
| $\{p, q, r\}$ | F | F | F |
| $\{p, q, \neg r\}$ | F | F | T |
| $\{p, \neg q, r\}$ | F | T | F |
| $\{p, \neg q, \neg r\}$ | F | T | T |
| $\{\neg p, q, r\}$ | T | F | F |
| $\{\neg p, q, \neg r\}$ | T | F | T |
| $\{\neg p, \neg q, r\}$ | T | T | F |
| $\{\neg p, \neg q, \neg r\}$ | T | T | T |

Table 1: $S_3$ with one clause removed. All 8 sets are *SAT* whits its satisfying interpretation.

## Case 2

How large can $n$ get? We run the solver on $S_n$, the $2^n$ clauses over $n$ propositional variables, increasing $n$ by one each time ($n > 3$). This shows how large an $n$ the solver can handle within a 60-second limit and an 8 GB memory cap. We ran the whole series 3 times and report the mean (Table 2).

**Observations.**

- From $n = 15$ to $n = 23$, each step multiplies the mean time by 1.9–2.4, so the time roughly doubles with each $n$. This matches the input, since the number of clauses doubles with each $n$.
- The step from n = 23 to n = 24 is larger, ×2.7. Running with +RTS -s shows a peak of 3.4 GB live data and 7.0 GB of heap in use, close to the 8 GB cap. Garbage collection took 71 of the 151 seconds of the whole run (47%), with single pauses of up to 7.6 s. So the extra slowdown at n = 24 comes from garbage collection, not from the search itself.
- Without a memory cap, an earlier version of the OS killed the process at n = 24 because it ran out of memory. With the cap, GHC solves the same problem within 8 GB.
- Times under 0.01 s vary a lot between runs and say little. The means for $n \geq 18$ are the meaningful ones.

Here is a table of the results for each of the 3 run and their means.
Every $S_n$ up to $n = 24$ was correctly found *UNSAT*.

| $n$ | Clauses | Run 1 (s) | Run 2 (s) | Run 3 (s) | **Mean (s)** | Result |
|---|---|---|---|---|---|---|
| 3 | 8 | 0.000 | 0.000 | 0.000 | **0.000** | UNSAT |
| 4 | 16 | 0.000 | 0.000 | 0.000 | **0.000** | UNSAT |
| 5 | 32 | 0.000 | 0.000 | 0.000 | **0.000** | UNSAT |
| 6 | 64 | 0.000 | 0.000 | 0.000 | **0.000** | UNSAT |
| 7 | 128 | 0.000 | 0.000 | 0.000 | **0.000** | UNSAT |
| 8 | 256 | 0.000 | 0.000 | 0.000 | **0.000** | UNSAT |
| 9 | 512 | 0.001 | 0.000 | 0.001 | **0.001** | UNSAT |
| 10 | 1,024 | 0.001 | 0.001 | 0.002 | **0.001** | UNSAT |
| 11 | 2,048 | 0.003 | 0.002 | 0.003 | **0.003** | UNSAT |
| 12 | 4,096 | 0.005 | 0.004 | 0.006 | **0.005** | UNSAT |
| 13 | 8,192 | 0.011 | 0.007 | 0.011 | **0.010** | UNSAT |
| 14 | 16,384 | 0.026 | 0.018 | 0.030 | **0.025** | UNSAT |
| 15 | 32,768 | 0.062 | 0.044 | 0.072 | **0.059** | UNSAT |
| 16 | 65,536 | 0.143 | 0.100 | 0.157 | **0.133** | UNSAT |
| 17 | 131,072 | 0.343 | 0.249 | 0.367 | **0.320** | UNSAT |
| 18 | 262,144 | 0.730 | 0.535 | 0.763 | **0.676** | UNSAT |
| 19 | 524,288 | 1.390 | 1.121 | 1.670 | **1.394** | UNSAT |
| 20 | 1,048,576 | 2.491 | 2.598 | 3.026 | **2.705** | UNSAT |
| 21 | 2,097,152 | 5.300 | 5.343 | 5.505 | **5.383** | UNSAT |
| 22 | 4,194,304 | 11.333 | 11.197 | 11.869 | **11.466** | UNSAT |
| 23 | 8,388,608 | 25.089 | 23.922 | 25.147 | **24.719** | UNSAT |
| 24 | 16,777,216 | 66.825 | 69.422 | 67.410 | **67.886** | UNSAT |

Table 2: Running time on $S_n$, three runs per $n$. All three runs stopped at $n = 24$, the first $n$ over the 60-second limit. The largest $n$ solved within the limit is $n = 23$.

# 5 Discussion

For case 1, the results show the expected behaviour. The solver finds the full set of 8 clauses unsatisfiable, and each set of 7 clauses satisfiable. In all eight cases, the model matches the prediction, and an independent verifier test confirms that it satisfies every remaining clause. Same $UNSAT/SAT$ was reached when the clauses was given as user input.

For case 2, the running time roughly doubles with each $n$, which matches the doubling of the input and the expected $n \cdot 2^n$ work. The largest $n$ solved within the 60-second limit is $n = 23$. The jump at $n = 24$ is larger than in earlier steps. At $n = 24$, the program needs 7 GB. Since the input doubles with every n, memory grows as fast as time. At $n = 25$, it would not fit in the 8 GB cap.

# 6 Conclusion

We implemented a DPLL-based SAT solver that uses unit propagation and atomic cut, and it meets the assignment requirements. The program runs on the two fixed test cases and user input. To verify the correctness of the SAT-solver, we uses a test suite that checks each rule on its own and compares the solver with brute force on all clause sets over atoms. Together with the independent verification of every model, this gives us confidence in the correctness of the result for case 1.

For case 2, every interpretation falsifies exactly one clause of $S_n$, so $S_n$ is unsatisfiable for every $n$, and the solver returned UNSAT in every run. The more interesting question is how large an $n$ the solver can handle. The largest $n$ solved within the 60-second limit is $n = 23$. Since the input doubles with each $n$, memory is the most likely limit: without a memory cap the process ran out of memory at $n = 24$, and with the cap that step was clearly slower than the earlier ones.